Projet tutoré de développement

Informatique — Licence 2 (L2)

Cours INF-L2-23

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Mener un projet d'application en équipe, du cahier des charges à la soutenance : choisir un sujet réaliste, organiser le travail, modéliser, réaliser par versions successives, tester, documenter, rédiger le rapport et défendre le projet devant un jury.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Choisir un sujet et rédiger le cahier des charges
• Leçon 2 — Constituer l'équipe et planifier le travail
• Leçon 3 — Modélisation et choix techniques argumentés
• Leçon 4 — Réalisation par étapes avec versions intermédiaires
• Leçon 5 — Tests, corrections et rédaction de la documentation
• Leçon 6 — Rédiger le rapport de projet
• Leçon 7 — Préparer la présentation orale du projet
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Le projet tutoré est souvent le premier « vrai » projet d'un étudiant en informatique : une équipe, un délai imposé, un encadrant qui joue le rôle du client, et au bout du chemin une application qui fonctionne, un rapport et une soutenance devant un jury. Tout ce que tu as appris dans les autres cours — programmation, bases de données, modélisation, génie logiciel — s'y rassemble en une seule expérience. Ce n'est plus un exercice corrigé en fin de chapitre : c'est toi qui dois décider, organiser et assumer tes choix. C'est aussi, très souvent, le premier projet que tu pourras montrer concrètement à un recruteur ou raconter en détail lors d'un entretien de stage.

La plupart des projets étudiants n'échouent pas sur la technique, mais sur l'organisation : sujet trop vaste découvert trop tard, équipe où une seule personne travaille, code commencé avant de savoir ce qu'il fallait construire, rapport écrit la veille de la soutenance. Ce cours te donne une méthode simple et éprouvée pour éviter ces pièges : cadrer avant de coder, découper le travail en versions intermédiaires, tester au fur et à mesure et écrire la documentation pendant le projet, pas après. Aucune de ces habitudes ne demande un talent particulier : elles demandent seulement d'être mises en place dès la première semaine, quand tout semble encore facile et que les délais paraissent lointains.

Nous suivrons pas à pas un projet fictif mais réaliste : GestPrêt, une application web de gestion des emprunts de livres et de matériel de la bibliothèque de l'IAM, réalisée par quatre étudiants de Licence 2 entre février et mai 2026. À chaque leçon, tu retrouveras la même équipe, les mêmes dates et les mêmes documents : le cahier des

charges, le planning, les choix techniques, les versions, les tests, le rapport et la soutenance. Tu pourras ainsi voir comment chaque document prépare le suivant, et pourquoi un retard pris en février se paie presque toujours en mai. En refaisant le même chemin avec ton propre sujet, tu auras toutes les cartes en main pour réussir ton projet tutoré.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Choisir un sujet de projet réaliste et le cadrer par un cahier des charges clair et validé.
- Constituer une équipe, répartir les rôles et construire un planning avec des jalons datés.
- Modéliser une application (cas d'utilisation, diagramme de classes, MCD) et justifier chaque choix technique.
- Réaliser une application par étapes, en livrant des versions intermédiaires testables.
- Organiser les tests, corriger les anomalies et rédiger une documentation utilisateur et technique.
- Rédiger un rapport de projet structuré, équilibré et relu, qui raconte le travail réellement accompli.
- Préparer et réussir une présentation orale chronométrée, démonstration et questions du jury comprises.

Leçon 1 — Choisir un sujet et rédiger le cahier des charges

Un bon sujet de projet tutoré tient en trois qualités : il répond à un besoin réel et identifiable, il est réalisable par ton équipe dans le temps imparti, et il mobilise les compétences du programme (une base de données, des interfaces, une logique métier). La bibliothèque de l'IAM gère ses emprunts sur des registres papier : les retards sont difficiles à suivre et les statistiques inexistantes. Voilà un besoin réel, proche, et dont les utilisateurs — les bibliothécaires — sont accessibles pour poser des questions. À l'inverse, « refaire un réseau social mondial » en dix semaines est un sujet condamné d'avance : trop vaste, impossible à tester, impossible à terminer.

Avant d'écrire quoi que ce soit, délimite le périmètre : ce que l'application fera, et surtout ce qu'elle ne fera pas. Cette frontière protège l'équipe des demandes qui s'ajoutent en cours de route. La méthode la plus simple consiste à lister les fonctions, puis à les classer : indispensables (le cœur du projet), souhaitables (si le temps le permet), et hors périmètre (écartées explicitement). Pour GestPrêt, la gestion des emprunts et des retours est indispensable ; les statistiques sont souhaitables ; le paiement en ligne des amendes est hors périmètre. Ce classement se fait avec l'encadrant, qui valide le sujet : un sujet validé tôt ne sera pas remis en cause à la soutenance.

Le cahier des charges transforme l'idée en contrat de travail. Il contient : le contexte et le problème, les objectifs, la description des utilisateurs, les fonctions attendues (numérotées, une phrase par fonction), les contraintes techniques et d'organisation, les livrables et les critères d'acceptation — c'est-à-dire comment on décidera que le projet est réussi. Chaque fonction doit être testable : « l'application doit être rapide » ne se teste pas ; « la recherche d'un livre doit afficher ses résultats en moins de deux secondes sur le poste de la bibliothèque » se teste. Le cahier des charges de GestPrêt tient en quatre pages : c'est suffisant, car son rôle n'est pas d'écrire un roman, mais de servir de référence commune à l'équipe, à l'encadrant et au jury.

L'essentiel
Un sujet réussi = un besoin réel + un périmètre délimité + une faisabilité vérifiée. Le cahier des charges est le contrat du projet : fonctions numérotées et testables, contraintes, livrables et critères d'acceptation validés par l'encadrant.

Exemple — Extrait du cahier des charges de GestPrêt (version validée le 13 février 2026)

```
CAHIER DES CHARGES — GestPrêt v1.0 (extrait)
Équipe : A. Ndiaye, M. Diarra, F. Sow, I. Kanté (L2 Info)
Encadrant : M. B. Traoré — Client : bibliothèque de l'IAM

1. CONTEXTE
La bibliothèque enregistre les emprunts sur des registres
papier : retards invisibles, statistiques impossibles.
GestPrêt remplacera les registres par une application web
utilisée sur le poste du bibliothécaire.

2. UTILISATEURS
- Bibliothécaire : seul utilisateur connecté (compte unique).
- Étudiants : consultés indirectement (reçus imprimés).

3. FONCTIONS (priorité : I = indispensable, S = souhaitable)
F1 (I) Enregistrer un emprunt (étudiant, ouvrage, date).
F2 (I) Enregistrer un retour et calculer les jours de retard.
F3 (I) Rechercher un ouvrage par titre ou par auteur.
F4 (I) Bloquer l'emprunt si l'étudiant a déjà 3 emprunts.
F5 (S) Tableau de bord : emprunts en cours, retards du jour.
F6 (S) Imprimer un reçu d'emprunt au format A5.
Hors périmètre : paiement des amendes, catalogue en ligne
public, application mobile.

4. CONTRAINTES
- Application web : PHP / MySQL, navigateur du poste actuel.
- Données de test fournies par la bibliothèque (200 ouvrages).
- Délai : version 1.0 livrée au plus tard le 8 mai 2026.

5. CRITÈRES D'ACCEPTATION
- F1 à F4 démontrées devant l'encadrant le 8 mai 2026.
- Aucune anomalie bloquante ouverte dans le cahier de tests.
- Manuel utilisateur de 4 pages maximum remis avec l'appli.
```

À retenir

• Le périmètre dit ce que le projet fait ET ce qu'il ne fait pas : les deux moitiés comptent.

• Une fonction bien écrite est numérotée, courte et testable — sinon elle créera des malentendus.

• Le cahier des charges est validé par l'encadrant avant de coder : c'est lui qui arbitrera les désaccords.

Exercice

Un groupe propose le sujet suivant : « Créer une application qui gère tout l'IAM : inscriptions, notes, emplois du temps, paiements, bibliothèque et cantine, en huit semaines, à trois étudiants. » 1) Cite trois raisons pour lesquelles ce sujet est irréaliste. 2) Propose un périmètre réduit et défendable. 3) Réécris la fonction floue « gérer les étudiants rapidement et facilement » en une exigence testable.

✔ Corrigé

1) Raisons : le périmètre couvre six domaines métier différents, chacun suffisant pour un projet entier ; trois étudiants en huit semaines ne peuvent ni modéliser ni tester tout cela ; les utilisateurs sont multiples (secrétariat, enseignants, étudiants) avec des règles métier que l'équipe ne connaît pas, donc impossible à valider sérieusement. 2) Périmètre réduit possible : conserver un seul domaine, par exemple la gestion des emprunts de la bibliothèque pour un seul utilisateur (le bibliothécaire), avec emprunts, retours, recherche et alertes de retard — c'est le choix fait pour GestPrêt. 3) Exigence testable : « Sur le poste de la bibliothèque, la recherche d'un étudiant par son numéro de carte doit afficher sa fiche et ses emprunts en cours en moins de deux secondes. » On a remplacé « rapidement et facilement » (incontrôlables) par une action précise, un contexte et un seuil mesurable.

Leçon 2 — Constituer l'équipe et planifier le travail

Un projet tutoré se fait en équipe de trois à cinq étudiants, et la répartition des rôles n'est pas une formalité : elle décide qui est responsable de quoi. Les rôles à couvrir sont : le chef de projet (planning, réunions, arbitrages), le responsable technique (choix d'outils, architecture, intégration), les développeurs, le responsable des tests et de la documentation. Une même personne peut porter deux rôles, mais chaque rôle doit avoir un nom en face. Dans l'équipe de GestPrêt, Awa est cheffe de projet et tient la documentation, Moussa est responsable du back-end et de la base de données, Fatou du front-end et des maquettes, Ibrahim des tests et de la mise en ligne. Quand un problème survient, on sait toujours qui s'en occupe.

Le planning découpe le projet en tâches datées, regroupées entre des jalons : des dates fixes où l'équipe doit livrer quelque chose de vérifiable. Un jalon n'est pas « avancer le code » mais « cahier des charges validé », « modélisation validée », « version 0.5 démontrée », « version 1.0 livrée », « rapport remis », « soutenance ». Pour GestPrêt, lancé le lundi 2 février 2026, les jalons tombent les vendredis : 13 février (cahier des charges), 6 mars (modélisation et maquettes), 17 avril (prototype 0.5), 8 mai (version 1.0 testée), 22 mai (rapport remis) et 29 mai (soutenance).

Construis le planning à rebours depuis la soutenance : la dernière semaine ne doit contenir aucun développement nouveau, seulement la relecture du rapport et la préparation orale.

Planifier ne suffit pas : il faut suivre. L'équipe de GestPrêt se réunit trente minutes chaque lundi : chacun dit ce qu'il a terminé, ce qu'il fera cette semaine et ce qui le bloque. Les décisions et les retards sont notés dans un compte rendu d'une demi-page, conservé dans le dossier du projet — ce journal servira plus tard à rédiger le rapport. Un retard vu dès la deuxième semaine se rattrape ; un retard découvert la veille du jalon devient une catastrophe. Enfin, prévoyez une marge : sur dix semaines de projet, gardez-en une sans tâche nouvelle, car il y aura toujours un imprévu (maladie, panne, fonction plus dure que prévu).

L'essentiel
Des rôles nommés, des jalons datés et un suivi hebdomadaire court : ce sont les trois piliers de l'organisation. Le planning se construit à rebours depuis la soutenance, avec une semaine de marge sans développement.

Exemple — Planning et répartition des tâches de l'équipe GestPrêt

```
PLANNING GestPrêt — fév. à mai 2026 (jalons = vendredis)

Période Tâches principales Responsable
02/02 - 13/02 Analyse, cahier des charges Awa (+ tous)
JALON 13/02 : CDC validé par M. Traoré
16/02 - 06/03 Cas d'utilisation, MCD, Moussa
maquettes Figma Fatou
JALON 06/03 : modélisation validée
09/03 - 17/04 Développement v0.1 à v0.5 Moussa, Fatou
(squelette, emprunts, retours) Ibrahim (tests)
JALON 17/04 : prototype 0.5 démontré
20/04 - 08/05 Fonctions restantes, tests, Tous
corrections, manuel utilisateur Awa, Ibrahim
JALON 08/05 : version 1.0 livrée
11/05 - 22/05 Rapport de projet, relectures Awa (+ tous)
JALON 22/05 : rapport remis
25/05 - 29/05 Diaporama, répétitions orales Tous
SOUTENANCE : vendredi 29 mai 2026

Règles de fonctionnement adoptées le 02/02 :
- Réunion d'équipe chaque lundi, 30 minutes, compte rendu noté.
- Code partagé sur GitHub, branche par fonctionnalité.
- Personne ne reste bloquée plus de 2 jours sans en parler.
```

À retenir

- Un jalon est un livrable vérifiable à date fixe, pas une intention de travailler.
- Le compte rendu hebdomadaire d'aujourd'hui devient la matière première du rapport de demain.
- La dernière semaine est réservée au rapport et à la soutenance : plus aucun code nouveau.

Exercice

Une équipe de quatre étudiants dispose de six semaines avant la soutenance. Son planning actuel est : « Semaines 1 à 5 : développer l'application. Semaine 6 : écrire le rapport et préparer la soutenance. » Identifie quatre défauts de ce planning, puis propose un squelette de planning corrigé avec des jalons.

✔ Corrigé
Défauts : 1) aucun jalon intermédiaire — le premier livrable vérifiable arrive trop tard pour corriger le tir ; 2) aucune phase d'analyse ni de modélisation avant le développement ; 3) les tests sont absents, ils seront donc bâclés ou sacrifiés ; 4) le rapport et la soutenance sont comprimés dans la même semaine que la fin du développement, sans marge pour les imprévus. Planning corrigé (jalons les vendredis) : semaine 1 — analyse et cahier des charges validé ; semaine 2 — modélisation et maquettes validées ; semaines 3-4 — développement avec version 0.5 démontrée fin de semaine 4 ; semaine 5 — tests, corrections, version 1.0 livrée, documentation ; semaine 6 — rapport relu et remis en début de semaine, répétitions de la soutenance. Chaque fin de semaine produit un livrable que l'encadrant peut vérifier.

Leçon 3 — Modélisation et choix techniques argumentés

Avant la première ligne de code, l'équipe modélise l'application : qui fait quoi (diagramme de cas d'utilisation), quelles données manipuler (modèle conceptuel de données), comment les écrans s'enchaînent (maquettes). Pour GestPrêt, le diagramme de cas d'utilisation montre un seul acteur, le bibliothécaire, et cinq cas : enregistrer un emprunt, enregistrer un retour, rechercher un ouvrage, consulter les retards, gérer le catalogue. Le modèle de données fait apparaître quatre entités : Étudiant, Ouvrage, Emprunt et Catégorie. Cette phase n'est pas une perte de temps : c'est là que l'équipe a découvert que la règle « trois emprunts maximum » devait être vérifiée au moment de l'enregistrement, et non à l'affichage — une erreur de conception détectée sur papier coûte une heure ; la même erreur dans le code en coûte dix.

Les choix techniques doivent être argumentés, pas seulement annoncés. Pourquoi PHP et MySQL plutôt qu'un autre couple ? Parce que l'équipe les a pratiqués en L1 et en L2, que l'hébergement de l'IAM les propose déjà, et qu'aucune installation n'est nécessaire sur le poste du bibliothécaire. Pourquoi des maquettes Figma avant de coder ? Parce qu'un écran validé par la bibliothécaire évite de refaire trois fois l'interface. Le jury posera presque toujours la question « pourquoi ce choix ? » : la bonne réponse cite un critère (compétence de l'équipe, coût, contrainte du client, simplicité) et reconnaît la limite du choix. Un choix assumé et expliqué vaut mieux qu'un choix « à la mode » impossible à défendre.

La modélisation se termine par une validation avec l'encadrant — c'est le jalon du 6 mars pour GestPrêt. Après cette date, le modèle ne change plus que pour corriger une erreur, jamais pour ajouter une fonction : c'est la règle du gel du périmètre. Si une bonne idée arrive en cours de développement, elle est notée dans une liste « évolutions possibles » qui alimentera la conclusion du rapport. Cette discipline protège

la date de livraison : l'équipe qui ajoute des fonctions en avril est la même qui découvre en mai qu'elle n'a rien testé. Modéliser, choisir, argumenter, puis geler : voilà le rythme qui transforme une idée en application terminée.

L'essentiel
Modéliser avant de coder (cas d'utilisation, données, maquettes), choisir chaque outil avec un argument défendable, puis geler le périmètre après validation. Les idées nouvelles vont dans la liste des évolutions, pas dans le planning.

Exemple — Tableau des choix techniques de GestPrêt et leurs arguments

```
CHOIX TECHNIQUES — GestPrêt (validés le 06/03/2026)

Besoin Choix retenu Argument principal
Langage serveur PHP 8 Pratiqué en cours L1/L2,
hébergement IAM compatible
Base de données MySQL (phpMyAdmin) Volumes faibles (200 livres),
outil connu de l'équipe
Interfaces HTML/CSS/JS Aucune installation sur le
simple poste du bibliothécaire
Maquettes Figma Validation par la cliente
avant de coder les écrans
Versionnement Git + GitHub Historique, travail à 4,
sauvegarde extérieure du code
Modélisation UML + MCD Vus en cours (INF-L2-06),
(draw.io) relus par l'encadrant

Choix écartés, avec motif :
- Application mobile Android : hors périmètre du CDC,
le besoin est sur un poste fixe de la bibliothèque.
- Framework lourd (type Symfony) : courbe d'apprentissage
incompatible avec un délai de dix semaines.
- Fichier Excel partagé : pas de contrôle des règles
métier (3 emprunts max) ni des accès simultanés.
```

À retenir

- Un choix technique sans argument est une opinion ; avec un critère et une limite reconnue, c'est une décision.
- Les diagrammes servent à découvrir les erreurs de conception quand elles coûtent encore une heure de papier.
- Après validation, le périmètre est gelé : les nouvelles idées rejoignent la liste des évolutions possibles.

Exercice

Pour le module « statistiques » de GestPrêt, un étudiant propose d'ajouter un graphique généré par une bibliothèque JavaScript externe plutôt qu'un simple tableau de chiffres. Rédige l'argumentaire complet de ce choix : deux arguments pour, une limite ou un risque, et la conclusion que tu défendrais devant le jury si on te demande « pourquoi ne pas avoir gardé le tableau ? ».

✔ Corrigé
Arguments pour : la lecture est immédiate pour la bibliothécaire (un retard qui grimpe se voit d'un coup d'œil, là où un tableau exige de comparer des lignes) ; la bibliothèque est libre, légère et ne demande aucune donnée envoyée à un serveur extérieur, donc compatible avec l'usage hors ligne du poste. Limite : c'est une dépendance de plus à intégrer et à tester, et le temps de chargement ou un conflit de version peut casser l'affichage — il faut donc prévoir un repli : le tableau reste affiché sous le graphique. Conclusion défendable : « Nous avons choisi le graphique pour la lisibilité, en gardant le tableau comme source de vérité et comme solution de secours ; si la bibliothèque pose un problème, l'application reste utilisable. » L'argumentaire est complet car il donne le bénéfice, le risque et la parade.

Leçon 4 — Réalisation par étapes avec versions intermédiaires

On ne construit pas une application d'un seul bloc : on la fait grandir par versions intermédiaires, chacune démontrant quelque chose de précis. La version 0.1 de GestPrêt est un squelette : les pages existent, la connexion à la base fonctionne, mais aucune règle métier n'est codée. La version 0.3 ajoute l'enregistrement des emprunts, la 0.5 — le prototype démontré le 17 avril — couvre F1 à F3, et la version 1.0 du 8 mai contient toutes les fonctions indispensables testées. Cette progression a deux vertus : l'équipe voit très tôt si elle est dans la bonne direction, et l'encadrant peut réagir sur du concret plutôt que sur des promesses.

Travailler à quatre sur le même code exige des règles : un dépôt Git partagé, une branche par fonctionnalité, et l'interdiction de pousser sur la branche principale un code qui ne fonctionne pas. Chaque version intermédiaire est étiquetée (tag) avec son numéro, de sorte qu'on peut toujours revenir en arrière. Ibrahim, responsable des tests, tient le journal des versions : date, contenu, problèmes connus. Ce journal est précieux à deux titres : il prouve au jury que le travail a été progressif — ce qu'il valorise — et il fournit la matière du chapitre « réalisation » du rapport. Une équipe qui ne peut pas montrer ses versions intermédiaires donne l'impression d'avoir tout fait en une nuit, même si ce n'est pas vrai.

Réaliser par étapes impose aussi de savoir dire « terminé » pour chaque fonction : une fonction est terminée quand elle marche sur le poste de démonstration, qu'elle gère les cas d'erreur prévus (par exemple un ISBN inexistant) et qu'un autre membre de l'équipe l'a essayée. C'est la définition du fini. Sans elle, toutes les fonctions restent « presque finies » et l'équipe découvre à la fin qu'aucune ne l'est vraiment. Le rythme tenu par GestPrêt — une démonstration à l'encadrant à chaque jalon — transforme ces « presque » en constats datés, bien avant qu'il ne soit trop tard pour corriger.

L'essentiel
Développer par versions numérotées et démontrables : squelette, fonctions indispensables, puis fonctions souhaitables. Git, branches par fonctionnalité, journal des versions et une définition stricte du « terminé » pour chaque fonction.

Exemple — Journal des versions intermédiaires de GestPrêt

```
JOURNAL DES VERSIONS — GestPrêt (tenu par I. Kanté)

Version Date Contenu livré État
0.1 20/03/2026 Squelette : pages, menu, OK
connexion MySQL, table livres
0.2 27/03/2026 Gestion du catalogue OK
(ajout, modification, liste)
0.3 03/04/2026 Enregistrement des emprunts OK
(F1) et des retours (F2)
0.4 10/04/2026 Recherche par titre/auteur OK
(F3), blocage 3 emprunts (F4)
0.5 17/04/2026 PROTOTYPE démontré à Validé
l'encadrant : F1 à F4 avec réserve*
0.6 24/04/2026 Tableau de bord (F5), OK
liste des retards du jour
0.7 01/05/2026 Impression du reçu A5 (F6), OK
corrections de la réserve*
1.0 08/05/2026 Version livrée : F1 à F6, Livrée
tests complets, manuel joint

* Réserve du 17/04 : le message d'erreur quand le livre
est déjà emprunté n'était pas clair. Corrigé en 0.7 :
« Ce livre est déjà emprunté par A. Sow, retour prévu
le 24/04/2026. »
```

À retenir

- Chaque version démontre un résultat précis ; aucune version ne doit rester « presque finie ».
- Le journal des versions documente la progression : c'est une preuve de travail pour le jury.
- Une fonction est terminée quand elle marche, gère ses erreurs et a été essayée par un autre membre.

Exercice

Une équipe présente à son encadrant, au bout de sept semaines, une application « terminée à 90 % » : tout est commencé, rien n'est démontrable séparément, et il reste « juste les tests et les détails ». Explique pourquoi ce « 90 % » est trompeur, puis décris comment cette équipe aurait dû découper son travail en versions 0.1 à 1.0 si son application comporte : la connexion des utilisateurs, la gestion des produits, la prise de commande et les statistiques.

✔ Corrigé
Le « 90 % » est trompeur parce qu'il mesure du code écrit, pas des résultats vérifiables : tant qu'aucune fonction complète n'a été démontrée et testée, le travail restant réel est inconnu — intégration, erreurs et cas limites se cachent précisément dans les derniers « détails », qui représentent souvent la moitié de l'effort. Découpage correct : v0.1 squelette + connexion des utilisateurs démontrée ; v0.3 gestion des produits complète (créer, modifier, lister) ; v0.5 prise de commande de bout en bout sur un produit — prototype démontrable ; v0.7 cas d'erreur et validations ; v0.9 statistiques ; v1.0 intégration complète testée, documentation jointe. Chaque version laisse l'équipe avec quelque chose qui fonctionne, même si le projet devait s'arrêter là.

Leçon 5 — Tests, corrections et rédaction de la documentation

Tester n'est pas « cliquer partout pour voir ». Un test sérieux part des fonctions du cahier des charges et, pour chacune, définit des cas : le cas normal (l'emprunt d'un livre disponible), les cas limites (l'étudiant a exactement trois emprunts, le retour le jour même), et les cas d'erreur (livre déjà emprunté, étudiant inconnu). Pour GestPrêt, Ibrahim a consigné ces cas dans un cahier de tests : pour chaque cas, l'action, le résultat attendu, le résultat obtenu et le verdict. Ce document transforme les discussions (« chez moi ça marchait ») en faits vérifiables, et il prouve au jury que la version 1.0 a été contrôlée, pas seulement espérée.

Les anomalies trouvées sont classées par gravité : bloquante (l'application plante ou perd des données), majeure (une fonction donne un résultat faux), mineure (un libellé confus, un alignement). On corrige d'abord les bloquantes, et chaque correction est suivie d'un re-test complet de la fonction touchée — car une correction peut en casser une autre : c'est le test de non-régression. Le critère de livraison de GestPrêt, écrit dans le cahier des charges, était « aucune anomalie bloquante ouverte » : à la livraison du 8 mai, il restait deux anomalies mineures, notées et assumées, ce qui est honnête et défendable. Cacher une anomalie connue est toujours plus grave que l'anomalie elle-même.

La documentation se rédige en deux volets. Le manuel utilisateur (quatre pages pour GestPrêt) explique à la bibliothécaire, avec des captures d'écran, comment faire chaque tâche et quoi faire en cas de message d'erreur ; il est testé par une personne extérieure à l'équipe — si elle réussit sans aide, le manuel est bon. La documentation technique, plus courte, décrit l'architecture, la structure de la base et la procédure d'installation, pour qu'un autre étudiant puisse reprendre le projet l'an prochain. Ces deux documents se rédigent pendant le projet, écran par écran, jamais la dernière semaine : ils alimentent directement le rapport et montrent au jury que l'application pourrait vivre sans ses auteurs.

L'essentiel
Tester = des cas écrits (normaux, limites, erreurs) avec résultats attendus et obtenus. Corriger par ordre de gravité, puis re-tester pour éviter les régressions. Documenter en continu : un manuel utilisateur testé par un tiers, une documentation technique pour reprendre le projet.

Exemple — Extrait du cahier de tests de GestPrêt (fonction F4 : blocage à 3 emprunts)

```
CAHIER DE TESTS — GestPrêt v1.0 (extrait, I. Kanté)
Fonction testée : F4 — blocage si 3 emprunts en cours

N° Cas testé Attendu Obtenu Verd.
T41 Étudiant avec 0 emprunt Emprunt Emprunt OK
en cours, livre libre accepté accepté
T42 Étudiant avec 2 emprunts, Emprunt Emprunt OK
3e emprunt accepté accepté
T43 Étudiant avec 3 emprunts, Refus + Refus + OK
4e demande message clair message
T44 Étudiant avec 3 emprunts, Emprunt Emprunt OK
dont 1 retourné ce matin accepté accepté
T45 Livre déjà emprunté par Refus + nom Refus OK*
un autre étudiant du détenteur sans nom
T46 Carte d'étudiant inconnue Message Message OK
(n° 9999) « inconnu »

* T45 : anomalie A-07 (majeure) — le nom du détenteur
actuel ne s'affichait pas. Corrigée le 02/05 (v0.7),
re-testée le 03/05 : OK. Anomalies restantes à la
livraison : A-11 et A-13 (mineures, libellés).
```

À retenir

- Les cas limites sont les plus instructifs : c'est là que vivent les erreurs (3 emprunts pile, retour le jour même).
- Toute correction impose un re-test de la fonction : corriger sans re-tester, c'est semer des régressions.
- Un manuel utilisateur n'est validé que si un tiers réussit à s'en servir sans l'aide de l'équipe.

Exercice

Pour la fonction F2 de GestPrêt (enregistrer un retour et calculer les jours de retard, sachant que le délai normal est de 14 jours), écris cinq cas de test complets (action, résultat attendu) en couvrant obligatoirement : un cas normal sans retard, un cas limite exact, et un cas d'erreur réaliste.

✔ Corrigé
Cas possibles : 1) Cas normal sans retard — emprunt du 2 mai, retour le 10 mai : 0 jour de retard, livre redevenu disponible. 2) Cas normal avec retard — emprunt du 2 mai, retour le 20 mai : 18 jours écoulés, donc 4 jours de retard affichés. 3) Cas limite — retour exactement le 14e jour (16 mai) : 0 jour de retard, car le délai est respecté à un jour près inclus ; c'est ce cas qui révèle si l'équipe a codé « supérieur à 14 » ou « supérieur ou égal ». 4) Cas limite lendemain — retour le 17 mai : exactement 1 jour de retard. 5) Cas d'erreur — tenter d'enregistrer le retour d'un livre qui n'est pas en cours d'emprunt : l'application doit refuser poliment avec un message clair, sans créer de ligne fantôme dans la base. Chaque cas précise la date d'emprunt, la date de retour et le résultat attendu : il est donc rejouable par n'importe quel membre de l'équipe.

Leçon 6 — Rédiger le rapport de projet

Le rapport de projet est la trace écrite et durable du travail : quand l'application ne sera plus installée nulle part, c'est lui que les enseignants reliront pour évaluer le projet. Sa structure classique comporte une introduction (le contexte, le problème, les objectifs), un chapitre d'analyse (besoin, cahier des charges, existant), un chapitre de conception (modèles, choix techniques argumentés), un chapitre de réalisation (versions, difficultés rencontrées et solutions), un chapitre de tests, puis un bilan honnête et une conclusion ouverte sur les évolutions possibles. Chaque chapitre correspond à une phase réellement vécue : c'est pour cela que le journal des versions, les comptes rendus de réunion et le cahier de tests, tenus depuis février, deviennent la matière première du rapport de GestPrêt, rédigé entre le 11 et le 22 mai.

Un bon rapport se reconnaît à trois qualités. L'équilibre : aucune partie ne doit écraser les autres — dix pages de code commenté et une demi-page d'analyse signalent un projet mal cadré. L'honnêteté : les difficultés et les renoncements (la fonction F6 a failli être abandonnée faute de temps) y ont leur place, car le jury fait la différence entre un rapport publicitaire et un rapport de travail. La lisibilité : figures numérotées et légendées, captures d'écran commentées, phrases courtes, et une orthographe relue — un rapport truffé de fautes discrédite silencieusement un contenu pourtant sérieux. Prévoyez deux relectures croisées : chacun relit les chapitres des autres, jamais les siens.

Le rapport se rédige en équipe mais avec une seule plume responsable de l'harmonie finale : chez GestPrêt, Awa assemble les contributions dans un document unique, avec une page de garde normalisée, une table des matières générée automatiquement et une bibliographie qui cite les sources réellement utilisées. Attention au piège du copier-coller documentaire : décrire une technologie en recopiant sa notice n'apporte rien ; le rapport doit raconter ce que VOUS avez fait avec elle. Dernière règle pratique : le rapport est remis quelques jours avant la soutenance (le 22 mai pour GestPrêt), donc sa rédaction commence dès la version 1.0 livrée, pas après — la dernière semaine appartient à l'oral.

L'essentiel
Le rapport raconte le travail réel, dans l'ordre des phases vécues : analyse, conception, réalisation, tests, bilan. Équilibre entre chapitres, honnêteté sur les difficultés, figures légendées, relectures croisées et remise avant la semaine de la soutenance.

Exemple — Plan détaillé du rapport de GestPrêt (28 pages hors annexes)

```
PLAN DU RAPPORT — GestPrêt (remis le 22/05/2026)

Pages de garde, remerciements, sommaire (3 p.)
Introduction générale (1 p.)
Contexte, problème des registres papier,
objectifs et annonce du plan
Chap. 1 — Analyse et cahier des charges (4 p.)
1.1 Étude de l'existant à la bibliothèque
1.2 Expression du besoin et utilisateurs
1.3 Fonctions F1 à F6 et critères d'acceptation
Chap. 2 — Conception (6 p.)
2.1 Diagramme de cas d'utilisation
2.2 Modèle conceptuel des données
2.3 Choix techniques argumentés
2.4 Maquettes des écrans principaux
Chap. 3 — Réalisation (6 p.)
3.1 Organisation de l'équipe et planning réel
3.2 Versions intermédiaires (journal)
3.3 Difficultés : la réserve du 17/04 et sa
correction ; l'anomalie A-07
Chap. 4 — Tests et validation (4 p.)
4.1 Méthode et cahier de tests
4.2 Résultats et anomalies restantes
Chap. 5 — Bilan et perspectives (2 p.)
5.1 Objectifs atteints / renoncements assumés
5.2 Évolutions : amendes, catalogue en ligne
Conclusion générale (1 p.)
Bibliographie et webographie (1 p.)
Annexes : manuel utilisateur, extraits de code,
comptes rendus de réunion (8 p.)
```

À retenir

- Le rapport prouve la démarche, pas seulement le résultat : il doit montrer l'analyse et les renoncements autant que le code.
- Équilibre et honnêteté sont les deux critères que le jury lit entre les lignes.
- Les documents produits pendant le projet (journal, tests, comptes rendus) sont la matière du rapport : rien à inventer.

Exercice

Le sommaire d'un rapport concurrent contient : « Chapitre 1 : le langage PHP (historique, syntaxe, avantages) — 12 pages. Chapitre 2 : notre application — 2 pages. Chapitre 3 : conclusion — 1 page. » Critique ce plan en quatre points précis, puis propose un plan rééquilibré pour ce même projet (une application de gestion d'une

cantine scolaire) en donnant le titre et le rôle de chaque chapitre.

✔ Corrigé
Critique : 1) le chapitre dominant décrit une technologie générique au lieu du travail de l'équipe — c'est du remplissage documentaire, pas un rapport de projet ; 2) l'analyse du besoin et la conception sont absentes, donc on ne sait ni pour qui ni pourquoi l'application existe ; 3) la partie « notre application », cœur du sujet, ne pèse que 2 pages sur 15 : le déséquilibre est flagrant ; 4) tests, organisation et bilan ont disparu, le jury ne peut évaluer ni la rigueur ni la démarche. Plan rééquilibré : Introduction (contexte de la cantine, problème des commandes papier, objectifs) ; Chap. 1 Analyse (utilisateurs : cantinière et élèves, fonctions, contraintes) ; Chap. 2 Conception (modèles, choix techniques argumentés en deux pages maximum, sans recopier de notice) ; Chap. 3 Réalisation (versions, écrans, difficultés) ; Chap. 4 Tests (cas, résultats, anomalies) ; Chap. 5 Bilan et perspectives ; Conclusion ; annexes (manuel, extraits de code). La technologie n'apparaît que là où elle sert le récit du projet.

Leçon 7 — Préparer la présentation orale du projet

La soutenance transforme quatre mois de travail en quinze minutes de parole suivies de questions : elle ne se prépare donc pas comme un résumé du rapport, mais comme une démonstration. Le jury a (ou aura) le rapport ; ce qu'il attend, c'est de voir l'application fonctionner et d'entendre l'équipe expliquer ses choix. La structure qui fonctionne : une accroche sur le problème (le registre papier et ses retards invisibles), le besoin et les utilisateurs, la solution en une phrase, la démonstration en direct de GestPrêt — le moment central —, puis la démarche (planning, versions, tests), les difficultés surmontées, et une conclusion qui ouvre sur les évolutions. Chaque membre parle sur la partie qu'il a réellement réalisée : le jury repère immédiatement quelqu'un qui récite le travail d'un autre.

La démonstration se prépare comme un scénario : un jeu de données propre et connu, un enchaînement d'actions répété (créer l'emprunt, provoquer le blocage des trois emprunts, enregistrer le retour en retard), et un plan B si le réseau ou le projecteur tombe en panne — captures d'écran ou courte vidéo de secours. Les diapositives restent sobres : un titre, trois idées maximum, une capture ou un schéma ; le texte complet est dans vos notes, pas à l'écran. L'équipe de GestPrêt a chronométré trois répétitions complètes : les durées ont guidé les coupes — c'est en répétant qu'on découvre qu'une explication prend trois minutes au lieu d'une.

Enfin, préparez les questions du jury au lieu de les craindre. Elles tournent presque toujours autour des mêmes thèmes : pourquoi ce choix technique plutôt qu'un autre, quelle est la limite principale de votre solution, qu'avez-vous testé exactement, que feriez-vous avec un mois de plus, et qui a fait quoi dans l'équipe. La bonne attitude : répondre précisément, reconnaître ce qu'on ne sait pas, et défendre ses choix avec les arguments du cahier des choix techniques — sans jamais inventer. Une équipe qui dit « cette fonction n'est pas couverte par nos tests, voici pourquoi et comment nous la testerions » marque plus de points qu'une équipe qui bluffe. La soutenance du 29 mai 2026 se joue donc largement avant d'entrer dans la salle.

L'essentiel
La soutenance = une démonstration scénarisée au centre, un discours chronométré et répété, des diapositives sobres, un plan B technique, et des réponses préparées aux questions classiques du jury. Chacun défend la partie qu'il a faite.

Exemple — Plan de soutenance chronométré de GestPrêt (15 minutes) et questions probables

```
SOUTENANCE GestPrêt — vendredi 29/05/2026 (15 min)

Min Orateur Contenu
0:00 Awa Accroche : le registre papier, un
retard découvert 3 semaines trop tard
1:30 Awa Besoin, utilisateurs, cahier des
charges et périmètre (F1-F6)
3:30 Fatou Maquettes puis DÉMONSTRATION en
direct : emprunt, blocage 3 livres,
retour en retard (scénario répété)
8:00 Moussa Conception : MCD, choix PHP/MySQL
argumentés, structure de l'appli
10:00 Ibrahim Démarche : planning, versions 0.1
à 1.0, cahier de tests, anomalie A-07
12:30 Awa Bilan honnête, limites, évolutions
(amendes, catalogue en ligne)
14:00 Tous Remerciements, ouverture aux
questions du jury

Questions probables préparées :
Q1 Pourquoi PHP et pas un langage plus récent ?
Q2 Que se passe-t-il si 2 postes utilisent l'appli
en même temps ? (limite assumée : poste unique)
Q3 Comment avez-vous testé la règle des 3 emprunts ?
Q4 Qui a écrit la fonction de calcul des retards ?
Q5 Avec un mois de plus, quelle fonction d'abord ?
Plan B : vidéo de la démonstration (3 min) sur clé
USB + captures d'écran dans le diaporama.
```

À retenir

- Le centre de la soutenance est la démonstration en direct : elle se répète comme un scénario, avec un plan B.
- On ne sait pas ce qu'on vaut à l'oral avant de s'être chronométré au moins trois fois.
- Reconnaître une limite avec sa parade prête vaut mieux qu'un bluff que le jury démasquera en une question.

Exercice

Pendant une répétition, l'équipe de GestPrêt constate : présentation totale de 19 minutes pour 15 autorisées ; la démonstration seule prend 7 minutes ; Awa parle 11 minutes à elle seule. Propose quatre corrections concrètes et chiffrées pour la répétition suivante, en respectant la structure imposée (démonstration au centre,

chacun parle de sa partie).

✔ Corrigé
1) Ramener la démonstration de 7 à 5 minutes en supprimant un scénario secondaire (l'impression du reçu F6, montrée en une capture à l'écran) et en gardant le triptyque emprunt / blocage / retour en retard. 2) Couper 2 minutes dans les parties discours : l'accroche passe de 2 minutes à 1 minute 30, et le chapitre conception perd l'explication détaillée du MCD — seuls les choix techniques argumentés, attendus par le jury, sont conservés. 3) Rééquilibrer la parole : Awa passe de 11 à 5 minutes (accroche, besoin, bilan), Moussa et Ibrahim prennent chacun une minute de plus sur leurs parties — le jury doit entendre les quatre membres sur leur propre travail. 4) Ajouter une règle de secours pendant l'oral : si à 10 minutes la démonstration n'est pas terminée, on passe directement au bilan, les points restants étant couverts par les réponses aux questions. Total visé : 14 minutes, avec une minute de sécurité.

Travaux dirigés (TD)

TD 1 — Transformer un besoin flou en cahier des charges testable

La responsable de la cantine scolaire de l'IAM écrit à une équipe d'étudiants : « Les élèves perdent du temps à la pause parce qu'ils choisissent leur plat au comptoir. On voudrait un petit système simple et moderne pour commander à l'avance, un truc rapide, fiable, que tout le monde peut utiliser, et qui nous aide aussi un peu pour la gestion. » 1) Relève dans ce texte toutes les expressions vagues ou non testables. 2) Liste les questions précises à poser à la responsable avant d'écrire le cahier des charges. 3) Rédige le début du cahier des charges : contexte, utilisateurs, cinq fonctions numérotées (dont deux indispensables) et deux critères d'acceptation mesurables.

✔ Corrigé du TD 1
1) Expressions vagues : « simple », « moderne », « rapide », « fiable », « que tout le monde peut utiliser », « qui nous aide aussi un peu pour la gestion » — aucune ne peut être vérifiée telle quelle, et « tout le monde » ne dit pas qui sont les utilisateurs. 2) Questions à poser : qui commande (élèves seuls ? le personnel aussi ?) ; sur quel support (téléphone des élèves, borne, poste de la cantine ?) ; jusqu'à quelle heure peut-on commander pour le jour même ? ; le paiement est-il inclus ou la commande est-elle seulement une réservation ? ; combien de plats et de menus par jour ? ; qui saisit les menus et qui peut annuler une commande ? ; que se passe-t-il si un plat est épuisé ? 3) Début de cahier des charges possible — Contexte : à la pause, les élèves choisissent au comptoir, la file dure 10 à 15 minutes et les plats populaires s'épuisent sans information. Utilisateurs : les élèves (commande), la cantinière (saisie des menus, liste des commandes). Fonctions : F1 (indispensable) consulter le menu du jour ; F2 (indispensable) commander un plat avant 10 h 30 et recevoir un numéro de commande ; F3 annuler sa commande avant 10 h 30 ; F4 afficher à la cantinière la liste des commandes par plat ; F5 (souhaitable) signaler un plat épuisé. Critères d'acceptation : commander un plat prend moins d'une minute sur le téléphone d'un élève ; à 10 h 45, la cantinière peut afficher le nombre exact de commandes par plat. Le passage du flou au testable s'est fait par des questions, pas par des suppositions.

TD 2 — Construire le planning d'un projet à partir de sa date de soutenance

Une équipe de trois étudiants (Sara : front-end ; Karim : back-end et base ; Léa : cheffe de projet, tests, documentation) démarre un projet de gestion des réservations de salles de l'IAM le lundi 9 mars 2026. La soutenance est fixée au vendredi 12 juin 2026. Les phases imposées : analyse et cahier des charges (2 semaines), modélisation (2 semaines), réalisation avec un prototype démontré à mi-parcours, tests et corrections (2 semaines), rapport (2 semaines), préparation orale (incluse dans la dernière semaine). Construis le planning complet : dates de chaque phase, six jalons datés sous forme de livrables vérifiables, et la répartition des responsabilités. Identifie ensuite deux risques de planning et la parade associée.

✔ Corrigé du TD 2
Planning (les jalons tombent les vendredis) : Analyse 09/03 → 20/03 : jalon 20/03, cahier des charges validé par l'encadrant (Léa, avec toute l'équipe). Modélisation 23/03 → 03/04 : jalon 03/04, cas d'utilisation, MCD et maquettes validés (Karim pour le MCD, Sara pour les maquettes). Réalisation première partie 06/04 → 01/05 : jalon 01/05, prototype 0.5 démontré couvrant les fonctions indispensables (réserver, annuler, consulter le planning des salles). Réalisation fin + fonctions souhaitables 04/05 → 15/05. Tests et corrections 18/05 → 29/05 : jalon 29/05, version 1.0 livrée avec cahier de tests sans anomalie bloquante (Léa aux tests, Karim et Sara aux corrections). Rapport 01/06 → 09/06 : jalon 09/06, rapport relu et remis (Léa assemble, tous rédigent leur partie). Préparation orale 08/06 → 12/06 (en parallèle léger de la fin du rapport) : trois répétitions chronométrées, soutenance le vendredi 12/06. Risques : 1) la semaine du prototype coïncide souvent avec des contrôles d'autres matières — parade : fonctions indispensables limitées et gel du périmètre dès le 03/04 ; 2) la rédaction du rapport déborde sur la préparation orale — parade : le journal de versions et les comptes rendus hebdomadaires sont tenus dès mars, et la remise du rapport est placée trois jours avant la soutenance, pas la veille.

Travaux pratiques (TP)

TP 1 — Dossier complet d'un mini-projet tutoré en quatre semaines

Par équipe de trois ou quatre, choisissez un besoin réel et proche (file d'attente du secrétariat, prêt de matériel de sport, gestion des absences en TD…) et produisez, en quatre semaines, le dossier complet d'un mini-projet tutoré, sans développer l'application entière. Semaine 1 : sujet cadré + cahier des charges de trois pages (fonctions numérotées, périmètre, critères d'acceptation) validé par l'enseignant. Semaine 2 : planning à rebours avec jalons + modélisation (cas d'utilisation, MCD, deux maquettes d'écran) + tableau des choix techniques argumentés. Semaine 3 : prototype cliquable ou squelette fonctionnel limité à une seule fonction indispensable, testé par les trois cas écrits (normal, limite, erreur) dans un cahier de tests. Semaine 4 : mini-rapport de six pages suivant le plan type du cours + soutenance blanche de huit minutes devant une autre équipe, avec grille d'évaluation remplie par l'équipe spectatrice.

✔ Attendus du TP 1
Attendus et grille d'évaluation (sur 20) : Cahier des charges (5 pts) : besoin réel documenté, fonctions numérotées et testables, hors-périmètre explicité — déduire 2 pts par fonction floue du type « rapide et facile ». Planning et modélisation (5 pts) : jalons datés et vérifiables, modèles cohérents entre eux (les fonctions du CDC se retrouvent dans les cas d'utilisation), choix techniques argumentés avec au moins un choix écarté et son motif. Prototype et tests (5 pts) : la fonction indispensable fonctionne en démonstration, les trois cas sont consignés avec résultats attendus et obtenus, les anomalies sont classées par gravité. Rapport et soutenance blanche (5 pts) : plan équilibré, honnêteté sur ce qui n'a pas été fait, démonstration au centre de l'oral, temps respecté à une minute près. Bonus collectif (jusqu'à +2) : journal de bord tenu chaque semaine sans trou. Le TP est réussi si le dossier, donné à une autre équipe, lui permet de comprendre le projet sans explication orale supplémentaire.

Exercices corrigés supplémentaires

Exercice 1

Dans l'équipe de GestPrêt, une semaine avant le jalon du prototype, Fatou (front-end) est hospitalisée trois jours et Moussa veut « tout reprendre » en travaillant seul la nuit, en sautant les réunions. Quels sont les risques de sa solution, et que doit faire Awa, la cheffe de projet, en tant que responsable du planning ? Propose un plan de rattrapage réaliste pour tenir le jalon du 17 avril.

✔ Corrigé
Risques de la solution de Moussa : épuisement et erreurs dans un code écrit fatigué, donc davantage d'anomalies à tester ; perte de traçabilité — le travail de nuit non partagé ne passe ni par Git ni par les comptes rendus ; rupture d'équipe, car Fatou retrouvera un code qu'elle ne reconnaît plus et Moussa deviendra le seul détenteur du savoir, exactement ce qu'un projet tutoré doit éviter. Actions d'Awa : 1) constater le retard officiellement dans le compte rendu (ne pas le cacher à l'encadrant — un jalon décalé négocié vaut mieux qu'un jalon raté surprise) ; 2) re-prioriser : le prototype 0.5 du 17 avril ne doit démontrer que F1 à F4 ; les écrans secondaires de Fatou sont simplifiés (formulaires sobres mais fonctionnels) plutôt que maquettés au pixel près ; 3) redistribuer temporairement : Ibrahim avance ses tests sur les fonctions déjà livrées, Awa prend en charge l'écran de consultation des retards, le plus simple ; 4) protéger la suite : la fonction F6 (impression) devient conditionnelle — elle ne sera intégrée que si la version 0.7 rattrape le retard, sinon elle bascule dans les évolutions du rapport. Le jalon est tenu sur le cœur du projet, sans sacrifier ni la santé de l'équipe ni la suite du planning.

Exercice 2

Classe les trois formulations suivantes de la meilleure à la moins bonne comme exigences d'un cahier des charges, en justifiant chaque rang : (a) « L'application doit être conviviale et rapide. » (b) « La liste des emprunts en retard doit pouvoir être affichée par le bibliothécaire. » (c) « Depuis l'écran d'accueil, le bibliothécaire affiche la liste des emprunts en retard en au plus deux clics, en moins de deux secondes sur le

poste de la bibliothèque. » Puis réécris la formulation (b) pour la porter au niveau de (c).

✔ Corrigé
Classement : (c) est la meilleure, car elle précise l'acteur, le point de départ, un nombre de clics et un temps mesurable dans un contexte donné — elle est directement transformable en cas de test. (b) vient ensuite : la fonction et l'utilisateur sont identifiés, mais ni le chemin d'accès ni la performance ne sont définis ; deux développeurs peuvent livrer deux résultats différents en prétendant tous deux avoir respecté l'exigence. (a) est la moins bonne : « conviviale » et « rapide » sont des appréciations subjectives, invérifiables, donc source de conflit à la validation. Réécriture de (b) au niveau de (c) : « Depuis n'importe quel écran, le bibliothécaire ouvre la liste des emprunts en retard via le bouton « Retards » de la barre de navigation (un clic), avec un affichage complet en moins de deux secondes pour 500 emprunts en cours. » On a ajouté le point d'accès, le nombre d'actions, la durée et le volume de données de référence.

Exercice 3

À la soutenance de GestPrêt, le jury pose trois questions pièges : (1) « Votre application ne gère qu'un seul utilisateur : est-ce vraiment un projet de Licence 2 ? » (2) « Qui a eu l'idée de la règle des trois emprunts, et qui l'a codée ? » (à poser à Fatou, qui n'a fait que les interfaces). (3) « Votre rapport dit que tout est testé, mais la page 24 parle de deux anomalies restantes : il y a une contradiction, non ? » Rédige la réponse que l'équipe doit donner à chaque question, en deux ou trois phrases, sans bluff et sans se contredire.

✔ Corrigé
(1) « C'est un choix de périmètre écrit et validé dans le cahier des charges : la bibliothèque fonctionne avec un poste unique, et nous avons préféré couvrir complètement le métier — emprunts, retards, blocages — plutôt que d'ajouter une gestion de comptes que personne n'utiliserait. La multi-utilisateur fait partie des évolutions listées dans le rapport. » La réponse transforme la limite en décision argumentée. (2) Fatou doit d'abord répondre pour son domaine puis passer la main proprement : « La règle vient du cahier des charges, à partir du règlement réel de la bibliothèque ; c'est Moussa qui l'a codée côté serveur, et moi qui ai conçu l'écran et le message de refus que vous avez vus dans la démonstration. Moussa peut vous détailler la vérification. » Aucun bluff : chacun défend sa partie, le jury obtient sa réponse. (3) « Il n'y a pas de contradiction : notre critère de livraison, écrit dans le cahier des charges, était “aucune anomalie bloquante ouverte”. Les deux anomalies restantes, A-11 et A-13, sont mineures — deux libellés de messages — documentées page 24 avec leur contournement. Nous avons préféré les déclarer plutôt que de prétendre à un zéro défaut invérifiable. » L'honnêteté documentée devient un argument de rigueur.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Le rôle principal du cahier des charges est de…

A) Remplacer le rapport de projet

B) Servir de contrat de référence : fonctions, périmètre, contraintes et critères d'acceptation

C) Choisir le langage de programmation à la place de l'équipe

2. Parmi ces exigences, laquelle est testable ?

A) Le système doit être rapide

B) Le système doit être moderne et agréable

C) La recherche d'un ouvrage affiche ses résultats en moins de deux secondes sur le poste de la bibliothèque

3. Un jalon de planning bien défini est…

A) Une réunion d'équipe chaque semaine

B) Un livrable vérifiable attendu à une date fixe

C) La fin du développement de toutes les fonctions

4. On construit un planning de projet tutoré…

A) À partir de la date de départ, en ajoutant les tâches dans l'ordre

B) Uniquement après avoir fini de coder

C) À rebours, depuis la date de soutenance, en réservant la fin au rapport et à l'oral

5. Après la validation de la modélisation, une idée de fonction nouvelle surgit. La bonne réaction est de…

A) L'ajouter discrètement au planning si elle semble rapide

B) La noter dans la liste des évolutions possibles et continuer le périmètre gelé

C) Recommencer la modélisation depuis le début

6. Une équipe défend son choix de MySQL devant le jury. L'argument le plus solide est…

A) « C'est le système le plus utilisé au monde. »

B) « C'est le système que l'encadrant préfère. »

C) « Nous l'avons pratiqué en cours, l'hébergement de l'institut le fournit, et nos volumes sont faibles. »

7. Une fonction peut être déclarée « terminée » quand…

A) Son code est écrit et poussé sur le dépôt

B) Elle fonctionne, gère les erreurs prévues et a été essayée par un autre membre

C) Le chef de projet l'a vue fonctionner une fois sur son poste

8. Dans un cahier de tests, les cas « limites » servent surtout à…

A) Tester les valeurs exactement aux frontières des règles (3 emprunts pile, retour le 14e jour)

B) Remplir le document pour atteindre le nombre de pages demandé

C) Tester l'application sur le plus vieil ordinateur disponible

9. Après la correction d'une anomalie majeure, il faut…

A) Passer immédiatement à la fonction suivante

B) Re-tester la fonction corrigée et ses voisines pour détecter les régressions

C) Réécrire tout le module depuis zéro par sécurité

10. Le rapport de projet doit principalement raconter…

A) L'histoire et la syntaxe des langages utilisés

B) Le travail réel de l'équipe : analyse, conception, réalisation, tests, bilan et renoncements

C) Uniquement les parties qui ont parfaitement fonctionné

Corrigés du QCM

1. Réponse B — Le cahier des charges cadre le travail et permet d'arbitrer les désaccords ; il ne remplace ni le rapport (qui raconte le travail accompli) ni les choix techniques (qui viennent après).

2. Réponse C — Seule la troisième donne une action, un contexte et un seuil mesurable : elle peut devenir un cas de test. Les deux autres sont des appréciations subjectives.

3. Réponse B — Un jalon sert à constater un résultat (« cahier des charges validé le 13 février »). Une réunion est un moyen de suivi, pas un livrable.

4. Réponse C — Le planning à rebours garantit que les phases non compressibles (rapport, soutenance) sont protégées, et que la dernière semaine ne contient plus de code nouveau.

5. Réponse B — Le périmètre gelé protège la date de livraison. L'idée n'est pas perdue : elle alimentera les « évolutions possibles » du rapport et de la conclusion.

6. Réponse C — Un choix technique se défend par des critères liés au projet : compétence de l'équipe, contraintes du contexte, adéquation au besoin. La popularité ou la préférence d'autrui ne sont pas des arguments.

7. Réponse B — La définition du fini évite les « presque terminés » : démonstration sur le poste de référence, cas d'erreur gérés, et vérification croisée par un pair.

8. Réponse A — Les erreurs se cachent aux frontières des règles : c'est là que les confusions entre « supérieur » et « supérieur ou égal » apparaissent.

9. Réponse B — Une correction peut casser autre chose : le re-test ciblé (test de non-régression) est obligatoire avant de déclarer l'anomalie close.

10. Réponse B — Le rapport prouve la démarche de l'équipe. La documentation des technologies n'y a sa place que si elle sert le récit, et les difficultés assumées y sont un signe de rigueur.

Résumé — les notions importantes

• Un sujet de projet tutoré réussi combine un besoin réel, un périmètre délimité (ce qu'on fait ET ce qu'on ne fait pas) et une faisabilité vérifiée avec l'encadrant.

• Le cahier des charges est le contrat du projet : fonctions numérotées et testables, contraintes, livrables et critères d'acceptation validés avant de coder.

• Organisation : des rôles nommés, un planning à rebours depuis la soutenance, des jalons-livrables datés et un suivi hebdomadaire consigné qui nourrira le rapport.

• Modéliser avant de coder (cas d'utilisation, données, maquettes), argumenter chaque choix technique par un critère, puis geler le périmètre : les idées nouvelles vont aux évolutions.

• Réaliser par versions intermédiaires numérotées et démontrables, avec Git, un journal des versions et une définition stricte du « terminé » pour chaque fonction.

• Tester par des cas écrits (normaux, limites, erreurs), corriger par gravité puis re-tester contre les régressions ; documenter en continu pour l'utilisateur et pour les repreneurs.

• Le rapport raconte honnêtement le travail réel et la soutenance est une démonstration scénarisée, chronométrée et répétée, avec plan B et réponses préparées aux questions du jury.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Comment rédiger un cahier de charges

Chaîne : AGLSoft Officiel ! · Langue : Français · Niveau : Débutant

Les rubriques indispensables d'un cahier des charges et la manière de les rédiger sans oublier les contraintes. À regarder avant d'écrire la première version du cahier des charges de ton équipe.

https://www.youtube.com/watch?v=Y56ZZToiUUI

Leçon 1 — Partie 1 Comment écrire un cahier de charges PRO (Méthode simple pour freelances développe #shorts

Chaîne : Konno Fuente · Langue : Français · Niveau : Débutant

Une méthode pas à pas pour structurer un cahier des charges professionnel, des fonctionnalités aux scénarios d'utilisation. Utile pour transformer un besoin flou en fonctions numérotées et testables.

https://www.youtube.com/watch?v=EJn46Xb3MGI

Leçon 2 — [Gestion de Projet] Leçon 3 : Initiation de projet et définition du contenu

Chaîne : Self Education · Langue : Français · Niveau : Intermédiaire

Le démarrage d'un projet vu côté gestion : charte, définition du contenu et place du cahier des charges fonctionnel. Un bon complément pour construire le planning et les jalons de ton équipe.

https://www.youtube.com/watch?v=QUeJ0qmN9DA

Leçon 7 — Comment réussir sa présentation [TP N°2]

Chaîne : Davy Landsi · Langue : Français · Niveau : Débutant

Sept conseils concrets pour réussir une présentation orale, académique ou professionnelle : simplicité, visuels, préparation et répétitions. À appliquer directement

au plan de soutenance chronométré de la leçon 7.

https://www.youtube.com/watch?v=aLLMSrYRG_I

Leçon 7 — Comment présenter un projet oralement : exercice d'empathie (extrait #1 du séminaire)

Chaîne : Stéphane Edouard · Langue : Français · Niveau : Intermédiaire

Un exercice qui démontre pourquoi une bonne présentation se construit du point de vue de l'auditoire, pas de l'orateur. Pertinent pour préparer un exposé de soutenance clair devant un jury qui découvre ton projet.

https://www.youtube.com/watch?v=3uu1H_7XyQM

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — règlement des projets tutorés et grille d'évaluation des soutenances à vérifier auprès du secrétariat pédagogique.

• Guide du Scrum (version française, scrumguides.org) — pour l'organisation d'une équipe en itérations courtes avec des livrables à chaque fin de cycle.

• Pressman, R. & Maxim, B. — « Software Engineering: A Practitioner's Approach » : les chapitres sur la gestion de projet et l'assurance qualité, pour approfondir planning et tests.

• Documentation Git (git-scm.com/doc) et guide GitHub (docs.github.com) — pour le versionnement des livrables intermédiaires du projet.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !